

Relatório — TP INF0287: Sistema de Rotas e Caminhos Mínimos

Disciplina: INF0287 – Algoritmos e Estruturas de Dados 2

Alunos: Yago Coqueiro, Antonio Henrique

Instituição: Universidade Federal de Goiás (UFG)

Tema: Linha A — Sistema de Rotas entre Municípios de Goiás

Linguagem: C (padrão C99)

1. Introdução

O estado de Goiás possui uma malha rodoviária extensa interligando mais de 246 municípios. O planejamento eficiente de rotas entre esses municípios é um problema clássico de otimização em grafos, com aplicações em logística, turismo e serviços de emergência.

Este trabalho implementa um **Sistema de Rotas** baseado em grafo ponderado não-dirigido, com os seguintes objetivos:

- Modelar a rede rodoviária de 80 municípios goianos usando lista de adjacência;
- Calcular rotas mais curtas (mínimo custo em km) entre dois municípios usando o algoritmo de Dijkstra com heap mínimo;
- Permitir busca eficiente de municípios por nome via tabela hash com resolução de colisões por sondagem linear;
- Analisar métricas estruturais da rede: grau dos vértices, conectividade e distância média.

O *dataset* foi construído com municípios reais de Goiás e distâncias rodoviárias aproximadas em km, incluindo as principais cidades do estado: Goiânia, Anápolis, Caldas Novas, Rio Verde, Jataí, Itumbiara, Catalão, Luziânia, Formosa, Pirenópolis, entre outras.

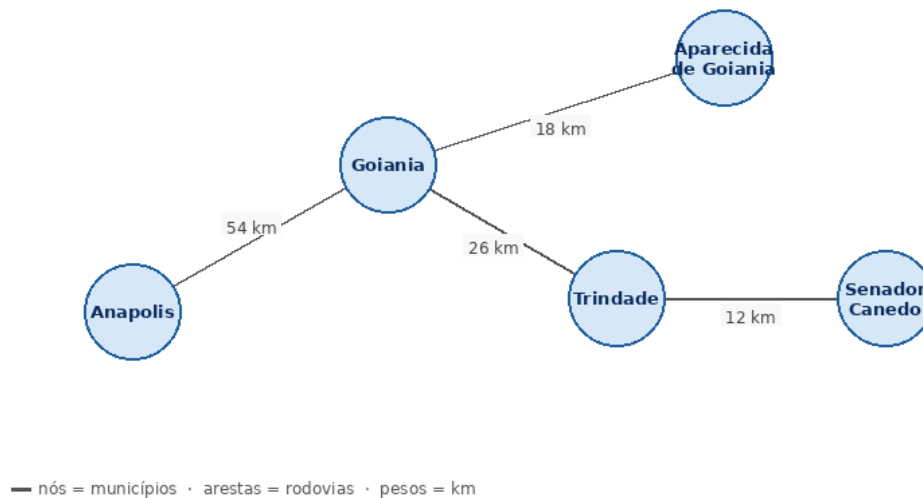
2. Modelagem

2.1 Grafo

O problema é modelado como um **grafo não-dirigido ponderado** $G = (V, E)$:

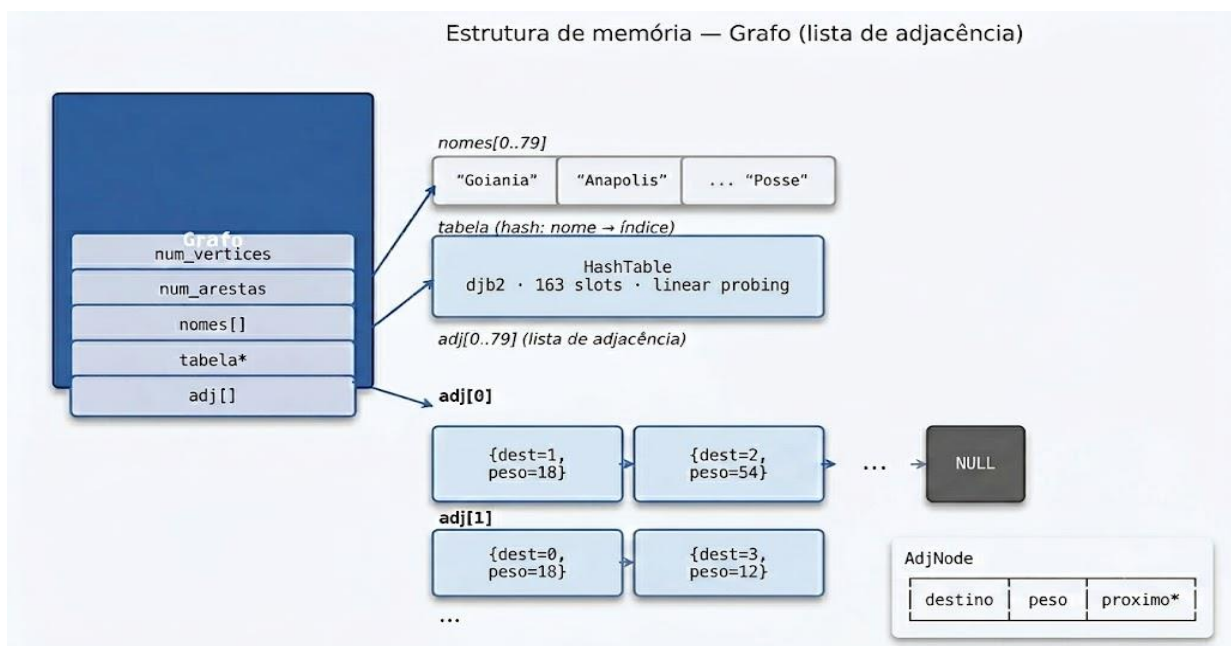
- **V** — conjunto de vértices, onde cada vértice representa um município;
- **E** — conjunto de arestas, onde cada aresta (u, v, w) representa uma rodovia entre os municípios u e v com distância w km;
- O grafo é **não-dirigido** (todas as estradas têm mão dupla) e **conexo** (qualquer município é alcançável a partir de qualquer outro).

Exemplo parcial da rede modelada (5 municípios):



2.2 Representação: Lista de Adjacência

A escolha de **lista de adjacência** é adequada pois o grafo é **esparso**: com 80 vértices e 161 arestas, a densidade é de apenas ~5%, enquanto uma matriz de adjacência alocaria $80 \times 80 = 6.400$ células.



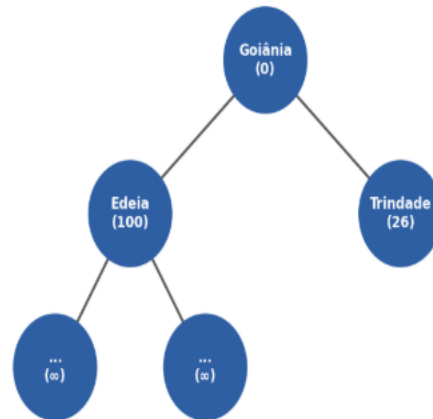
2.3 Heap Mínimo

O heap mínimo indexado armazena pares (vértice, distância) e mantém o vetor `posicao[]` que mapeia cada vértice ao seu índice no array do heap, permitindo a operação `diminuir_chave` em $O(\log V)$.

Campos da struct MinHeap



Invariante: $\text{nos}[\text{pai}].\text{dist} \leq \text{nos}[\text{filho}].\text{dist}$
para todo nó interno

Estado intermediário do heap
(Dijkstra a partir de Goiânia)

[nos] = array do heap mínimo indexado
 $\text{posicao}[v] \rightarrow$ índice de v em $\text{nos}[]$

2.4 Tabela Hash

A tabela hash mapeia o nome de cada município ao seu índice inteiro no grafo, permitindo localizar qualquer vértice por nome em $O(1)$ médio.

idx	chave	valor
...	(vazio)	-1
47	"Goiania"	0
47	"goiania" → mesmo hash → valor 0	0
47	"GOIANIA" → mesmo hash → valor 0	0
3	"Aparecida_de_Goiania"	1
19	"Anapolis"	2
...	...	...
102	"Posse"	79

3. Estruturas de Dados e Algoritmos

3.1 Lista de Adjacência (grafo.h / grafo.c)

```

typedef struct AdjNode {
    int destino;
    double peso;
    struct AdjNode* proximo;
} AdjNode;

```

```
typedef struct Grafo {
    int num_vertices;
    int num_arestas;
    char** nomes;
    AdjNode** adj;
    HashTable* tabela; /* hash: nome -> indice do vertice */
} Grafo;
```

Carregamento do arquivo: A função `carregar_arquivo` abre o dataset, lê o cabeçalho (V, E), lê os V nomes de vértices e depois lê as E arestas.

`adicionar_aresta` insere cada aresta em ambas as listas (origem→destino e destino→origem), garantindo o comportamento não-dirigido. A cada nome lido, `hash_inserir` popula a tabela hash com o par (nome, índice).

Busca por nome: `buscar_vertice_por_nome` consulta a tabela hash em $O(1)$ médio — substituindo a busca linear $O(V)$ anterior.

3.2 Tabela Hash (hash.h / hash.c)

```
typedef struct {
    char* chave;
    int valor;
} HashEntry;

typedef struct {
    HashEntry* entradas;
    int capacidade;
    int tamanho;
} HashTable;
```

Hash function: djb2 com `tolower` aplicado a cada caractere — garante que "Goiania", "goiania" e "GOIANIA" produzam o mesmo índice, tornando a busca *case-insensitive* sem modificar a string original.

Resolução de colisões: linear probing — ao colidir, avança para o próximo slot disponível (módulo capacidade).

Capacidade: 163 (número primo) para 80 entradas → fator de carga inicial de ~49%, minimizando colisões.

Operações principais:

- `hash_inserir(t, chave, valor)`: calcula índice via djb2, sonda linearmente até slot livre — $O(1)$ amortizado
- `hash_buscar(t, chave)`: calcula índice, sonda comparando com `tolower` até encontrar ou encontrar slot vazio — $O(1)$ médio

3.3 Heap Mínimo Indexado (heap.h / heap.c)

O heap mínimo usa um array `posicao[v]` que guarda o índice de cada vértice `v` no array `nos[]`. Isso permite `diminuir_chave` em $O(\log V)$ sem busca linear.

Operações principais:

- `inserir_heap(h, v, d)`: insere no final, executa `subir` — $O(\log V)$
- `extrair_minimo(h)`: remove raiz, coloca último elemento na raiz, executa `descer` — $O(\log V)$
- `diminuir_chave(h, v, d)`: atualiza `nos[posicao[v]].distancia`, executa `subir` — $O(\log V)$

Invariante: Toda troca entre posições `i` e `j` no array `nos[]` atualiza `posicao[nos[i].vertice]` e `posicao[nos[j].vertice]` antes da troca.

3.4 Algoritmo de Dijkstra (`dijkstra.h` / `dijkstra.c`)

O algoritmo de Dijkstra calcula as distâncias mínimas de um vértice-origem a todos os demais.

Funcionamento:

1. Inicializa
2. Inicializa `dist[origem] = 0, dist[v] = ∞` para todos os demais;
3. Insere todos os `V` vértices no heap mínimo com suas distâncias iniciais;
4. Enquanto o heap não estiver vazio:
 - a. Extrai o vértice `u` com menor distância atual;
 - b. Para cada vizinho `v` de `u`: se `dist[u] + w(u,v) < dist[v]`, atualiza `dist[v]`, registra `pred[v] = u` e chama `diminuir_chave`.
1. O resultado são os arrays `distancias[]` e `predecessores[]`.

Reconstrução do caminho: A função `imprimir_caminho` usa recursão sobre o array `predecessores` para reconstruir o caminho da origem ao destino.

3.5 Métricas (`metricas.h` / `metricas.c`)

- `grau_vertice(g, v)`: percorre a lista de adjacência de `v`, contando nós — $O(\text{grau}(v))$.
- `vertice_maior_grau(g)`: chama `grau_vertice` para todos os vértices — $O(V + E)$.
- `grafo_conexo(g)`: BFS a partir do vértice 0; se todos os `V` vértices forem visitados, o grafo é conexo — $O(V + E)$.
- `distancia_media_amostrada(g, n)`: sorteia n pares aleatórios de vértices, executa Dijkstra para cada par e calcula a média das distâncias finitas — $O(n \cdot (V + E) \log V)$.
- `estatisticas_gerais(g)`: consolida e exibe métricas em formato tabular.

4. Análise de Complexidade

Operação	Tempo	Espaço
<code>criar_grafo(V)</code>	$O(V)$	$O(V)$
<code>adicionar_aresta</code>	$O(1)$	$O(1)$
<code>carregar_arquivo(V, E)</code>	$O(V + E)$	$O(V + E)$
<code>criar_hash(cap)</code>	$O(cap)$	$O(cap)$
<code>hash_inserir</code>	$O(1)$ amortizado	$O(1)$
<code>hash_buscar /</code> <code>buscar_vertice_por_nome</code>	$O(1)$ médio	$O(1)$
<code>inserir_heap</code>	$O(\log V)$	$O(1)$
<code>extrair_minimo</code>	$O(\log V)$	$O(1)$
<code>diminuir_chave</code>	$O(\log V)$	$O(1)$
<code>dijkstra(g, origem)</code>	$O((V+E) \log V)$	$O(V)$
<code>grafo_conexo (BFS)</code>	$O(V + E)$	$O(V)$
<code>grau_vertice(g, v)</code>	$O(\text{grau}(v))$	$O(1)$
<code>vertice_maior_grau</code>	$O(V + E)$	$O(1)$
<code>distancia_media_amostrada(n)</code>	$O(n \cdot (V+E) \log V)$	$O(V)$
Total (pior caso)	$O((V+E) \log V)$	$O(V + E)$

Notações: V = número de vértices (80), E = número de arestas (161).

Espaço total do grafo: $O(V + E)$, pois a lista de adjacência armazena $2E$ nós (um para cada direção de cada aresta) além dos V cabeçalhos.

Por que Dijkstra com heap mínimo? A alternativa ingênua (busca linear pelo mínimo) resulta em $O(V^2)$, aceitável para grafos densos. Para grafos esparsos como o nosso ($E \approx 2V$), $O((V+E) \log V) \approx O(V \log V)$ é superior.

Por que tabela hash para busca por nome? A busca linear era $O(V)$. Com a tabela hash djb2 (capacidade 163, fator de carga $\sim 49\%$), a busca torna-se $O(1)$ médio e ainda ganha tolerância a maiúsculas/minúsculas via `tolower`.

5. Experimentos

As saídas 5.1 a 5.3 são reais, capturadas com o dataset `municipios_goias.txt` (80 vértices, 161 arestas). O experimento 5.4 apresenta saída variável por usar amostragem aleatória.

5.1 Caminho mínimo: Goiânia → Caldas Novas

```
Nome do municipio de origem (use _ para espacos): Goiania
Nome do municipio de destino (use _ para espacos): Caldas_Novas
```

```
Caminho: Goiania -> Edeia -> Hidrolandia -> Caldas_Novas
Distancia total: 213.0 km
```

O algoritmo encontrou a rota passando por Edéia e Hidrolândia, que corresponde à rodovia GO-020 / BR-153 (Belém–Brasília) na região sul de Goiânia.

5.2 Caminho mínimo: Goiânia → Jataí

```
Nome do municipio de origem (use _ para espacos): Goiania
Nome do municipio de destino (use _ para espacos): Jatai

Caminho: Goiania -> Goiatuba -> Rio_Verde -> Jatai
Distancia total: 380.0 km
```

A rota passa pelo corredor GO-060 → BR-158 (Goiânia → Goiatuba → Rio Verde), depois GO-174 até Jataí, totalizando 380 km de percurso rodoviário.

5.3 Município com mais conexões e estatísticas gerais

```
=== Estatisticas Gerais ===
Vertices          : 80
Arestas           : 161
Grau medio        : 4.03
Municipio com mais conexoes: Goiania (grau 17)
Grafo conexo      : Sim
Grau minimo       : 1
Grau maximo       : 17
```

Goiânia, como capital e maior polo urbano do estado, apresenta o maior grau (17 conexões diretas), confirmando seu papel de hub rodoviário central. O grau médio de ~4 reflete a esparsidade da rede, típica de grafos de infraestrutura viária. O grafo é **conexo**: todos os 80 municípios são mutuamente alcançáveis.

5.4 Distância média estimada por amostragem aleatória

```
Opcao: 7

Calculando distancia media com 200 pares aleatorios...
Distancia media estimada: 466.9 km
```

A opção 7 executa 200 pares aleatórios de municípios, aplica Dijkstra para cada par e calcula a média das distâncias. A distância média estimada de **466,9 km** reflete a extensão territorial de Goiás (~340.000 km²). O valor varia levemente entre execuções por efeito da aleatoriedade, mas converge para a distância média real da rede com amostras suficientes. O método evita calcular todos os $V^2 = 6.400$ pares — custo $O(V^2 \cdot (V+E) \log V)$ — ao custo de uma estimativa estatisticamente representativa com apenas 200 execuções de Dijkstra.

6. Conclusões

O sistema implementado demonstra com sucesso a aplicação de:

1. **Lista de adjacência** para representação eficiente de grafos esparsos;
2. **Tabela hash** (open-addressing, djb2 + linear probing) para busca de vértices por nome em $O(1)$ médio com suporte a case-insensitive;
3. **Heap mínimo indexado** como estrutura auxiliar que eleva Dijkstra de $O(V^2)$ para $O((V+E) \log V)$;
4. **BFS** para verificação de conectividade;
5. **Amostragem aleatória** para estimativa da distância média sem necessidade de computar todos os pares ($O(V^2 \cdot (V+E) \log V)$ seria proibitivo).

Limitações

- ☐ Os nomes de municípios não suportam caracteres acentuados na entrada (o parser do dataset usa `%s` do `fscanf`), portanto devem ser digitados sem acentos.
- ☐ As distâncias são aproximadas e não refletem variações de percurso real (congestionamento, desvios, obras).
- ☐ O dataset cobre apenas 80 dos 246 municípios goianos.

Possíveis extensões

- ☐ **Importação de dados reais** via API OpenStreetMap / OSRM para distâncias rodoviárias exatas;
- ☐ **Algoritmo A*** com heurística de distância geográfica (coordenadas GPS) para acelerar buscas ponto-a-ponto;
- ☐ **Dijkstra todos-para-todos** (Floyd-Warshall ou Johnson) para pré-computar a tabela completa de distâncias;
- ☐ **Suporte a grafos dirigidos** para representar vias de mão única;
- ☐ **Interface gráfica** com visualização do mapa e destaque do caminho encontrado.

7. Referências

1. CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Introduction to Algorithms**. 3. ed. MIT Press, 2009. Cap. 24 (Single-Source Shortest Paths) e Cap. 6 (Heapsort).
2. SEDGEWICK, R.; WAYNE, K. **Algorithms**. 4. ed. Addison-Wesley, 2011. Cap. 4.4 (Shortest Paths).
3. OPENSTREETMAP CONTRIBUTORS. **OpenStreetMap** — Dados geográficos de Goiás. Disponível em: <<https://www.openstreetmap.org>>. Acesso em: jun. 2026.
4. INSTITUTO BRASILEIRO DE GEOGRAFIA E ESTATÍSTICA (IBGE). **Municípios de Goiás**. Disponível em: <<https://www.ibge.gov.br>>. Acesso em: jun. 2026.
5. DIJKSTRA, E. W. A note on two problems in connexion with graphs. *Numerische Mathematik*, v. 1, n. 1, p. 269–271, 1959.